Estudante: Haniel Botelho Ribeiro (2023028552)
Curso: Ciências da Computação
Professora: Olga Nikolaevna Goussevskaia
Disciplina: Algoritmos I
Turma: TN
Trabalho Prático 2: Siderúrgica Metalmax

1. Introdução:

A Metalmax, uma das principais siderúrgicas especializadas na produção de aço, opera com uma rede elétrica interna altamente complexa para sustentar seus processos industriais, como fornos, laminadores e sistemas auxiliares. Essa rede é composta por geradores, máquinas consumidoras e conexão entre eles. Recentemente, a empresa enfrentou desafios significativos no fornecimento de energia, o que impactou negativamente sua produção e aumentou os custos operacionais. Então ela decidiu solicitar o nosso serviço como cientistas da computação.

Portanto, a empresa nos informou que seriam fornecidos dados sobre a quantidade de máquinas e de conexões na rede. Para cada unidade seria informado o identificador e o consumo de energia dela, um consumo igual a 0 significa que a unidade é um gerador. Além disso, para cada conexão é informado o identificador das unidades de origem e de destino e sua capacidade máxima de energia suportada.

Este trabalho propõe o desenvolvimento de um algoritmo capaz de analisar a estrutura e o desempenho da rede elétrica da Metalmax, oferecendo um diagnóstico detalhado sobre seu funcionamento: Calculando a capacidade máxima da rede, a energia faltante para os consumidores, a energia desperdiçada ao longo da rede e identificando conexões críticas. A fim de que pudéssemos apontar oportunidades de melhoria para otimizar a eficiência energética e reduzir os custos.

2. Modelagem:

Para representar a rede elétrica da Metalmax, utilizamos a abstração de um grafo direcionado com vértices e arestas. Essa escolha é ideal para capturar a relação entre geradores e consumidores de energia, bem como as capacidades das conexões entre eles.

Elementos do Grafo

Os vértices representam unidades da rede, podendo ser geradores ou consumidores, e são armazenados em um map<int, Vértice>, onde a chave é o identificador único do vértice. A estrutura Vértice contém uma lista de adjacência representada por um map<int, Aresta>, o que facilita a busca e manutenção das conexões, mantendo-as ordenadas pelo identificador do destino.

As arestas, por sua vez, representam conexões entre os vértices, com capacidade máxima e fluxo atual. Cada aresta é encapsulada em uma estrutura Aresta, que armazena a capacidade máxima e o fluxo efetivo.

O grafo é representado por uma classe que organiza os vértices em um map<int, Vértice> e incorpora vértices artificiais chamados superfonte e superdreno para consolidar geradores e consumidores, simplificando o cálculo de fluxo máximo. A classe também mantém atributos como a energia exigida (soma total de energia necessária pelos consumidores), energia atendida (quantidade de energia suprida pelo fluxo), energia fornecida (soma das capacidades das conexões dos geradores)

e uma lista de conexões críticas (tuplas com origem, destino e capacidade das arestas em sua capacidade máxima).

Estruturas de Dados Utilizadas

Os map<int, Aresta> e map<int, Vértice> garantem acesso eficiente e ordenação, facilitando as operações do algoritmo. Já as tuplas (int, int, int) representam as conexões críticas, identificando origem, destino e capacidade das arestas.

Razões para Escolhas de Modelagem

Optamos por uma lista de adjacência devido à sua eficiência espacial e facilidade de iteração. A inclusão de vértices artificiais simplifica o problema ao consolidar múltiplos geradores e consumidores, enquanto o uso de atributos pré-calculados, como energia exigida e atendida, evita cálculos redundantes e melhora o desempenho geral.

3. **Solução:**

A solução é dividida em etapas, iniciando pela instanciação do grafo. O construtor da classe Grafo recebe o número de vértices e arestas, criando os vértices superiores (superfonte e superdreno) para consolidar os geradores e consumidores. Durante a leitura dos dados, cada vértice é categorizado como gerador (consumo zero) ou consumidor (consumo maior que zero), com as respectivas conexões criadas.

Após essa etapa, aplicamos o algoritmo de Ford-Fulkerson para determinar o fluxo máximo. A busca por caminhos aumentantes é realizada por uma BFS que encontra caminhos da superfonte ao superdreno, considerando arestas com capacidade residual positiva. O fluxo do caminho é calculado como a menor capacidade residual, e as atualizações são feitas nas arestas e suas reversas. Esse processo é repetido até que nenhum caminho aumentante seja encontrado.

Com o fluxo máximo calculado, a energia faltante é determinada subtraindo a energia atendida da energia exigida. A energia perdida é calculada percorrendo as arestas conectadas aos geradores e subtraindo a energia atendida da energia fornecida.

Para identificar as conexões críticas, percorremos as arestas do grafo verificando se a capacidade utilizada é igual à máxima. Essas arestas são armazenadas como tuplas (origem, destino, capacidade) e ordenadas em ordem decrescente de capacidade com o uso de MergeSort.

Resumo da Solução

A solução consiste em construir o grafo com vértices artificiais e conexões, aplicar o algoritmo de Ford-Fulkerson para calcular o fluxo máximo e extrair métricas como energia faltante, energia perdida e conexões críticas. A abordagem é eficiente e modular, permitindo que cada etapa seja realizada de forma independente e reutilizando resultados intermediários sempre que necessário.

4. **Análise de Complexidade:**

A estrutura do grafo armazena V vértices, cada um contendo uma lista de adjacências, no total E arestas. Assim, o custo espacial é O(V+E). Adicionalmente O(E) para armazenar as arestas críticas, Θ(1) para atributos simples como energia atendida, exigida e fornecida. A estrutura de dados map tem um custo O(log n) para busca. A construção do grafo passa por definir alguns atributos e fazer a leitura e inserção dos vértices e arestas tem custo O(V+E).

Ford Fulkerson utiliza uma bfs, para encontrar o menor caminho aumentante, ela tem complexidade O(V+E). Já o Ford Fulkerson tem complexidade O(E*f), onde f é o fluxo máximo. Apesar de que a energia exigida é calculada conforme a entrada,

depois subtrai o fluxo máximo, vamos assumir um custo O(V). Para o cálculo de energia perdida é preciso percorrer todas as arestas geradoras e somando a capacidade delas e subtraí o fluxo máximo, complexidade O(E). Para encontrar as conexões críticas é preciso percorrer todas as arestas do grafo, complexidade O(E) e depois ordenar elas utilizando um MergeSort que tem complexidade O(n log n).

5. Considerações Finais:

A modelagem proposta para a rede elétrica da Metalmax, utilizando a abstração de grafos direcionados, mostrou-se eficaz para representar e resolver o problema de fluxo máximo de energia. A utilização de vértices artificiais, como a superfonte e o superdreno, simplificou o gerenciamento dos geradores e consumidores, enquanto a aplicação do algoritmo de Ford-Fulkerson permitiu calcular de forma eficiente o fluxo máximo, atendendo às demandas energéticas da rede.

Além disso, a identificação de conexões críticas e o cálculo de métricas como energia faltante e energia perdida forneceram informações valiosas para o diagnóstico e otimização da rede. A abordagem modular adotada garante flexibilidade para futuras expansões e adaptações do sistema, como a inclusão de novos geradores ou consumidores e a análise de cenários de falhas.

Como última consideração, temos que a escolha específica de cada ponto determinou uma saída específica, em outras palavras, uma modelagem diferente poderia chegar em outros resultados. Mais explicitamente, foi feito um teste utilizando unorderd_map que tem um acesso constante, apesar do fluxo máximo ser o mesmo, teve um caminho diferente, por não ser ordenado. Uma DFS poderia ter um resultado igual por caminhos diferentes, entre outras mudanças que podem ser feitas. Trabalhos como esse nos mostram, na prática, como a escolha da modelagem pode impactar no desempenho.

6. Referências:

https://cplusplus.com/reference/map/map/
https://cplusplus.com/reference/tuple/
https://www.geeksforgeeks.org/ford-fulkerson-algorithm-for-maximum-flow-problem/
https://www.geeksforgeeks.org/merge-sort/